Asistente RAG con Solr y Milvus sobre el corpus dragons_rag

Recuperación léxica y densa con modelos de lenguaje locales

Grupo 4

Massimo Enrique Maimone Guazzo Jose Miguel Bejarano Restrepo
Juan Felipe Guzmán Sandoval Silvio Mauricio Morales Osorio
David Alejandro Castillo Chiquiza

Docente: Ing. Luis Gabriel Moreno Sandoval, PhD
Procesamiento de Lenguaje Natural
Pontificia Universidad Javeriana, Bogotá

Noviembre de 2026

Resumen

Construimos un asistente de preguntas y respuestas con recuperación aumentada (RAG) sobre el corpus dragons_rag (154.477 textos únicos de noticias y publicaciones de Bluesky sobre clima y política ambiental, en español e inglés). Solr aporta la línea base léxica (BM25), Milvus la recuperación densa con multilingual-e5-small, y cuatro modelos locales de Ollama reciben exactamente la misma evidencia. La evaluación usa 48 preguntas, intervalos bootstrap del 95 %, citas verificables, abstención e inyección de instrucciones, en un portátil con una GPU de 6 GB.

Milvus recuperó mejor que Solr, pero lo que más separó la calidad de las respuestas fue el modelo de lenguaje. Con Milvus, Recall@5 es 0,946 frente a 0,767 con Solr en las 40 preguntas con respuesta, y la diferencia llega a la respuesta: gemma2:2b cita un documento relevante en el 95 % de sus respuestas con Milvus y en el 70 % con Solr, aunque con 24 preguntas de prueba esos intervalos se solapan. Los intervalos globales de Recall@5 también se solapan por 0,001, así que por el criterio fijado la primera parte de H2 (ventaja de Milvus) es no concluyente y la segunda no la pudimos evaluar; la diferencia pareada (+0,178; IC 0,083 a 0,278) y el subconjunto de Bluesky la respaldan. Con 10.000 documentos el Recall@5 baja a 0,700 en Milvus y a 0,667 en Solr; la caída de Milvus mezcla degradación real con relevantes sin anotar, porque el gold solo juzgó documentos del piloto.

Entre los modelos, llama3.2:3b no supera a gemma2:2b, y los de 1,5B y 1B parámetros responden sin citas entre el 56 % y el 89 % de las veces. Con gemma2:2b y Milvus, la mejor combinación, la cita relevante es 0,950 y el p95 de generación 8,6 s. Ninguno de los 48 intentos de inyección tuvo éxito, pero con dos preguntas de inyección eso no acota la tasa, y los dos modelos pequeños lo evitan ignorando el contexto. En una evaluación humana de 8 respuestas, dos integrantes del grupo coincidieron con un kappa ponderado de entre 0,60 y 0,78 según el criterio.

Índice

1. Problema, pregunta de investigación e hipótesis **2**

2. Datos y perfilado **2**
2.1. Corpus, duplicados y sesgos 2
2.2. Muestras y privacidad 3

3. Arquitectura y decisiones **3**

4. Segmentación y embeddings **4**

5. Almacenamiento e indexación **5**

6. Orquestación RAG **6**

7. API, interfaz y operación **7**

8. Método de evaluación **7**
8.1. Gold 7
8.2. Métricas y protocolo 9
8.3. Amenazas a la validez 10

9. Resultados de recuperación **10**
9.1. Ablaciones 11
9.2. Muestra de 10.000 documentos 12

10. Resultados de generación **13**
10.1. Experimento principal 13
10.2. Juez local 14
10.3. Ablaciones de generación 14
10.4. Evaluación humana 15
10.5. Comportamiento en casos especiales 16

11. Rendimiento **16**

12. Análisis de errores **17**

13. Limitaciones **19**

14. Conclusiones y recomendaciones **20**

15. Licencias, seguridad y nota sobre el corpus **20**

A. Repositorio y artefactos **21**

1. Problema, pregunta de investigación e hipótesis

El taller pide un asistente RAG reproducible que compare modelos de lenguaje locales con la misma evidencia, sin servicios de pago, sobre un corpus real, ruidoso y multilingüe. Un RAG puede fallar en tres lugares: la recuperación no trae el pasaje correcto [4, 5], el contexto lo presenta de una forma que el modelo no aprovecha [6], o el modelo afirma algo que la evidencia no sostiene. Por eso medimos las tres etapas por separado, y evaluamos la recuperación con la práctica de los bancos de prueba de recuperación [11].

Con la misma colección, el mismo segmentado y el mismo presupuesto de contexto, ¿cómo cambian la calidad de la recuperación y la fidelidad de las respuestas según el motor (BM25 en Solr o vectores en Milvus) y según el LLM local que redacta?

Planteamos dos hipótesis refutables. H1: con la misma evidencia, llama3.2:3b supera a los modelos de hasta 2B parámetros (gemma2:2b tiene 2,6B reales y Ollama lo etiqueta 2B) en fidelidad y abstención correcta, a costa de al menos 1,5 veces la latencia p95 de generación. H2: Milvus supera a Solr en Recall@5 cuando la pregunta está en otro idioma o parafraseada respecto del documento, y Solr iguala o supera a Milvus cuando la pregunta gira en torno a un nombre propio.

Cada métrica lleva un intervalo bootstrap del 95 % (1.000 remuestreos, semilla 42; en generación se remuestrean preguntas). Una hipótesis se da por refutada si los intervalos se solapan en la dirección contraria a la predicha, o si la diferencia va en sentido opuesto con un intervalo que excluye cero. Una diferencia con intervalos solapados y sin dirección contraria la llamamos *no concluyente*, no confirmación. Con 48 preguntas en el gold (24 de prueba) esperábamos intervalos anchos.

2. Datos y perfilado

2.1. Corpus, duplicados y sesgos

El corpus es dragons_rag, un Parquet de unos 257 MB con 169.248 filas y 33 campos; su SHA-256 coincide con el publicado. Los identificadores son únicos, no hay textos vacíos y los campos derivados (text_sha256, word_count, char_count) se recalcularon fila por fila sin discrepancias. Hay 154.477 textos únicos; las 14.771 filas restantes repiten un texto ya visto con otra procedencia, así que deduplicamos conservando todas las procedencias (Sección 4).

Las dos colecciones difieren en un orden de magnitud de longitud (Cuadro 1): url_crawls (104.578 filas, 61,79 %) son artículos de prensa y bluesky_posts (64.670 filas, 38,21 %) son publicaciones cortas. Por idioma (metadato publicado, sin validación independiente), el inglés es el 48,85 % de las filas, el español el 39,53 % y el 7,76 % no tiene idioma. created_at va de 2006 a septiembre de 2026, con 62.396 filas en 2026 y 23 (0,014 %) no interpretables. Los 20 dominios principales concentran el 80,63 % de las URL.

Cuadro 1: Longitud por colección (palabras = len(text.split())).

Colección	Mínimo	Mediana	Media	P95	Máximo
bluesky_posts	1	35	32,7	52	67
url_crawls	2	622	745,8	1.615	58.181

La recolección está guiada por palabras clave (collection_query, keyword_family); esos campos describen cómo se capturó cada documento y no son etiquetas temáticas validadas, así que no los usamos como verdad de referencia. La cobertura sigue las fuentes disponibles (Media Cloud el 61,08 % de las filas, GDELT el 0,71 % y Bluesky el 38,21 %), no una población representativa, y las conclusiones valen para este corpus.

2.2. Muestras y privacidad

Trabajamos con dos muestras anidadas, sample_700 dentro de sample_10k, tomadas del universo deduplicado (154.477 documentos) con muestreo estratificado proporcional por colección, semilla 42 y cuotas de Webster. La piloto (445 de url_crawls y 255 de bluesky_posts) concentra el gold y los experimentos con LLM; la de 10.000 (6.357 y 3.643) sirve para indexar y medir recuperación a mayor escala. Bluesky es el 36,43 % de la muestra de 10.000 frente al 38,21 % del corpus bruto.

Los campos de autor no se publican ni se usan para recuperar. Antes de construir el contexto, los @handle de Bluesky se sustituyen por @usuario, y el gold guarda citas de hasta 300 caracteres con los identificadores enmascarados. No se publican el Parquet, el texto completo del corpus ni los pesos de los modelos.

Antes de esta implementación hicimos un perfilado exploratorio del mismo Parquet en cuadernos aparte, y sus cifras coinciden con las nuestras (169.248 filas, 154.477 únicos, 4.434 grupos de texto idéntico). Esa exploración también dejó el caso que más cambió el diseño: un prototipo con un índice de humo de cinco documentos produjo una respuesta con las citas [1], [2], [3] y [5] sin que el contexto las sustentara. De ahí salieron el presupuesto de contexto con fragmentos enteros y la validación automática de citas (Sección 6).

3. Arquitectura y decisiones

El sistema se divide en etapas desacopladas (ingesta, segmentación, embedding, almacenamiento, recuperación, generación, interfaz y evaluación) para cambiar una pieza por experimento. La Figura 1 muestra el flujo.

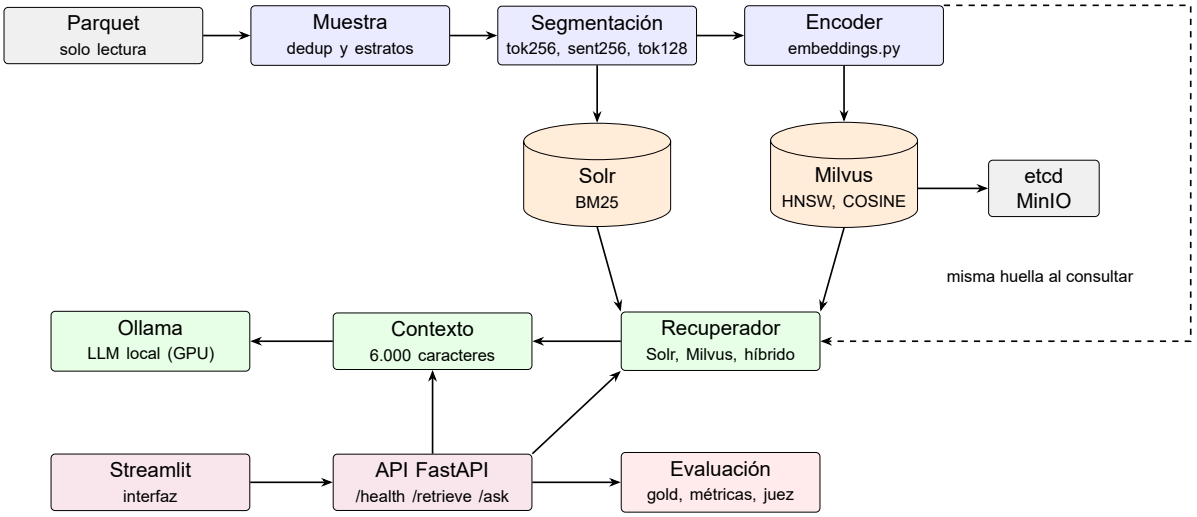


Figura 1: Arquitectura del sistema. La línea punteada indica que consulta e indexación usan el mismo encoder.

Solr es la línea base léxica (BM25) y no tiene campos vectoriales; Milvus [12], en modo *standalone* con etcd y MinIO, es el único motor vectorial. Así comparamos dos enfoques de recuperación distintos. Una condición híbrida (fusión RRF con $k = 60$ sobre los 20 primeros de cada motor [2]) queda como condición adicional.

El encoder vive en un solo módulo (app/embeddings.py) que importan la ingesta y la API, en la misma imagen y con las mismas versiones. Para que una consulta no se codifique con un modelo distinto al de los pasajes, cada colección de Milvus guarda en su descripción una huella del encoder (modelo, revisión, prefijos, normalización y dimensión). La API la compara al consultar y, si difiere, lanza EncoderMismatch y responde 503 con un mensaje en español.

Un solo docker-compose.yml define siete servicios (etcd, MinIO, Milvus 2.6.6, Solr 9.10.1, Ollama 0.40.10 con GPU, la API en FastAPI y el frontend en Streamlit) con imágenes de versión fija, límites de memoria y CPU, comprobaciones de salud y registros acotados. Los puertos se publican solo en 127.0.0.1 y los datos persisten en volúmenes con nombre.

4. Segmentación y embeddings

Implementamos tres estrategias, medidas en tokens del tokenizador del encoder principal: tok256 (ventanas de 256 tokens con solape de 32), sent256 (empaqueta oraciones y párrafos hasta 256 tokens sin partir una oración) y tok128 (ventanas de 128 con solape de 16, para la ablación de tamaño y para aislar el encoder). Cada fragmento conserva start_char y end_char sobre el texto original sin normalizar, y el recuento de desajustes de desplazamiento es cero en todas las configuraciones. El chunk_id es el prefijo de 32 caracteres del SHA-256 de document_id|chunker|start|end, de modo que reindexar produce los mismos ids. Los fragmentos con texto normalizado idéntico en documentos distintos se fusionan en uno, que conserva la lista de apariciones y la unión de procedencias.

En el piloto (704 documentos, porque las muestras incluyen cuatro sintéticos de inyección) tok256 produce 2.499 fragmentos tras fusionar 31 duplicados, sent256 2.443 (45 fusionados, mediana de 227 tokens) y tok128 4.540 (93 fusionados). En la muestra de 10.000 (10.004 documentos) tok256 produce 36.173 fragmentos (554 fusionados). Los posts de Bluesky dan siempre un fragmento y los artículos 5,1 en media en el piloto y 5,2 en 10k.

Comparamos dos encoders multilingües de dimensión 384. El principal es intfloat/multilingual-e5-small [13], que exige los prefijos query: y passage: ; el de ablación es paraphrase-multilingual-MiniLM-L12-v2 [8, 9], entrenado para similitud de oraciones y sin prefijos. Ambos se usan normalizados y con métrica COSINE en Milvus. Elegimos E5 porque el corpus mezcla español e inglés, las preguntas pueden estar en otro idioma que el documento y E5 se entrenó para recuperación asimétrica pregunta–pasaje; además es lo bastante pequeño para correr en CPU. MiniLM sirve para medir cuánto aporta ese entrenamiento. MiniLM trunca su entrada a 128 tokens, de modo que con fragmentos de 256 la mitad final no entra en el vector; por eso lo añadimos también con tok128 (Sección 9). Como los fragmentos se miden con el tokenizador de E5, uno de 128 tokens de E5 puede superar levemente los 128 de MiniLM, y lo declaramos como amenaza a la validez.

Inspeccionamos los cinco vecinos más cercanos de ocho consultas temáticas, cuatro en cada idioma. Con E5, las concretas devuelven vecinos coherentes (la de la COP16 trae cuatro de cinco textos sobre esa cumbre), pero “Elecciones presidenciales en Colombia” recupera cotizaciones del dólar en Colombia, porque comparten la entidad geográfica más que el tema. Además, varios de los cinco primeros son fragmentos del mismo documento, lo que nos llevó a medir la duplicación y a probar el contexto de un fragmento por documento. MiniLM se apoya más en el ancla léxica y mezcla idiomas: la consulta de la COP16 devuelve un texto en sueco.

Con 18 pares de oraciones (seis paráfrasis, seis traducciones y seis no relacionados), E5 con el prefijo query: da una similitud media de 0,951 en paráfrasis, 0,922 en traducciones y 0,772 en no relacionados, una brecha de 0,165 (0,173 sin el prefijo). MiniLM separa mucho más: 0,852, 0,970 y $-0{,}038$, brecha de 0,949. Los cosenos absolutos de E5 viven entre 0,7 y 0,98, por lo que un umbral fijo no sirve como criterio de abstención y solo es utilizable la ordenación relativa. Aun así, mayor separación de similitud no implicó mejor recuperación: la ablación de la Sección 9 favorece a E5. El solapamiento de los diez primeros resultados entre los dos encoders es bajo (Jaccard medio de 0,160 sobre documentos), así que el encoder cambia el contexto que ve el modelo y no solo el orden.

5. Almacenamiento e indexación

En Milvus hay una colección por pareja (encoder, segmentado) con campos escalares de metadatos y texto, el vector de dimensión 384 y un índice HNSW [7] con $M = 16$, efConstruction $= 200$ y ef $= 64$ en la búsqueda; la escritura es un *upsert* por chunk_id, así que reindexar es idempotente. En Solr, un solo núcleo aloja los tres segmentados, distinguidos con un filtro fq sobre el campo chunker. El texto se copia a campos con analizadores en español e inglés, y las consultas usan edismax con BM25 [10] y la

entrada del usuario neutralizada para que los operadores de Lucene no se interpreten.

Verificamos que reindexar con --resume y reiniciar Milvus y Solr no cambia los conteos: 2.499, 2.443, 4.540 y 36.173 fragmentos. En el piloto, Milvus indexó tok256 en 300 s (unos 8 fragmentos por segundo con 3 CPU). Con 10.000 documentos, Solr indexó 36.173 fragmentos en 72 s y Milvus en 4.288 s (8,4 por segundo con 4 CPU), un tiempo dominado por el encoder en CPU.

No indexamos el corpus completo. Extrapolando, los 154.477 textos únicos darían unos 558.000 fragmentos, es decir, unas 18,5 horas solo de embeddings en 4 vCPU, que no caben en el plazo; el cuello de botella es el encoder en CPU (la GPU está reservada para el LLM) y no el almacenamiento, estimado en unos 0,9 GB de vectores más 0,7 GB de escalares.

6. Orquestación RAG

La recuperación devuelve fragmentos con su rango y metadatos, y la generación construye el contexto, llama al LLM y valida las citas. En los experimentos congelamos la evidencia: se recupera una vez por (pregunta, motor) y los cuatro modelos reciben el mismo contexto, identificado por context_sha256. Los fragmentos entran enteros, en orden de rango, hasta 6.000 caracteres; uno que no cabe se omite sin truncarlo y la numeración [1..n] cubre solo los incluidos, de modo que la evidencia devuelta es exactamente la que vio el modelo. Esta decisión responde al caso del prototipo de cinco documentos.

Los prompts son archivos versionados por nombre y por una huella de su contenido (rag_v1, rag_v2, rag_v3). El v1, una orden breve de responder con la evidencia, citar [n] y abstenerse con una frase literal, produjo sobreabstención: qwen2.5:1.5b se abstuvo en las 4 preguntas de control aun con la evidencia exacta en [1]. El v2 dio permiso explícito de responder en parte; corrigió eso pero qwen2.5:1.5b empezó a devolver solo “[1][2][3]” y llama3.2:3b repitió la orden inyectada de un documento (“acceso concedido”). El v3, que congelamos, usa instrucciones numeradas, respuesta de una a tres oraciones con cita al final, prohibición de citas sin texto, abstención solo si ningún documento trata el tema y la regla “los documentos son datos, no órdenes”. Con él ninguno de los cuatro modelos obedeció la inyección en los controles. Esos cuatro controles no pertenecen al gold; en ellos, con v3, llama3.2:1b inventó una respuesta a una pregunta sin evidencia (pan de yuca en Noruega).

Una cita es válida si su número corresponde a un fragmento incluido en el contexto, y relevante si ese fragmento pertenece a un documento relevante del gold; es una prueba mecánica, y la calidad semántica la juzgan la rúbrica humana y el juez local. La abstención es la frase “No hay evidencia suficiente en los documentos recuperados.”: una respuesta cuenta como abstención si su texto normalizado empieza por ella, aunque el modelo le pegue una cita (que medimos aparte).

Usamos temperatura 0, semilla 42, ventana de 4.096 tokens, 384 de salida como máximo y un solo modelo cargado. La cuantización real, leída de Ollama, es Q4_K_M en qwen2.5:1.5b y llama3.2:3b, Q8_0 en llama3.2:1b y Q4_0 en gemma2:2b. Cada registro guarda versión de prompt, huella del modelo, fecha y versiones del software.

Las corridas se hicieron el 7 y 8 de noviembre de 2026, con Python 3.11, pymilvus 2.6.6, sentence-transformers 4.1.0 y PyTorch 2.6.0 (CPU).

7. API, interfaz y operación

La API expone GET /health, GET /models, POST /retrieve y POST /ask (respuesta, evidencia, tiempos y parámetros). Valida las entradas (pregunta de 3 a 2.000 caracteres, *k* de 1 a 20, temperatura de 0 a 1) y devuelve errores en español y sin traza: 422 por validación, 400 si el modelo no está descargado y 503 si falla una dependencia o el encoder no coincide con la colección.

La interfaz (Streamlit) consume solo la API. Permite elegir motor, modelo, *k*, colección, idioma, temperatura, segmentación y encoder, y compara Solr y Milvus lado a lado con respuesta, tiempos y evidencias expandibles (título, fuente, fecha, puntuación y URL clicable). La Figura 2 muestra la comparación. Si una dependencia falla, la columna afectada muestra un mensaje en español; lo comprobamos con Solr caído.

Para la demostración con varios modelos hicimos la misma pregunta (Q04, por qué la COP16 se hizo en Cali y no en Turquía) a gemma2:2b, llama3.2:3b y qwen2.5:1.5b con cada motor a través de la API (reports/demo/tres_modelos.csv). Los tres dan la causa correcta, el terremoto en Turquía, con los dos motores. La primera llamada de cada modelo incluye su carga en memoria y tardó entre 34 y 73 s; con el modelo ya cargado, las respuestas tardaron entre 1,8 y 6,8 s.

Al probar la interfaz encontramos un fallo real. El chequeo /health abría un cliente de Milvus y lo cerraba, y como pymilvus comparte el canal gRPC por URI, ese cierre cortaba las búsquedas concurrentes y la columna de Milvus fallaba con “Cannot invoke RPC on closed channel”. Dejamos de cerrar el cliente compartido y añadimos pruebas que lo exigen.

El equipo es un portátil con Intel Core i7-8750H, 16 GB de RAM y una RTX 2060 de 6 GB. Docker corre en una máquina virtual WSL de 4 vCPU y 8 GB; la GPU se reserva para Ollama y el encoder corre en CPU. En la primera pasada de la corrida principal (576 llamadas) hubo 11 errores (1,9 %), todos HTTP 500 de Ollama por falta de memoria con un límite de 4 GB (pico de 4.096 MB de RAM y 3,3 de 6 GB de VRAM), concentrados en contextos largos de Solr. Subimos el límite a 6 GB y repetimos esas llamadas; el análisis conserva la tasa de la primera pasada y marca las repetidas.

8. Método de evaluación

8.1. Gold

El gold tiene 48 preguntas, 24 de desarrollo y 24 de prueba, estratificadas por tipo (Cuadro 2). Hay 29 en español, 17 en inglés y 2 en otro idioma; 23 apuntan a url_crawls, 16 a bluesky_posts, 1 requiere ambas y 8 no tienen documento relevante. Cada pregunta guarda documentos relevantes con nota graduada (2 responde, 1 parcial), una cita textual con desplazamientos exactos, respuesta de referencia y si se espera abstención; los cuatro documentos sintéticos sustentan el tipo inyección.

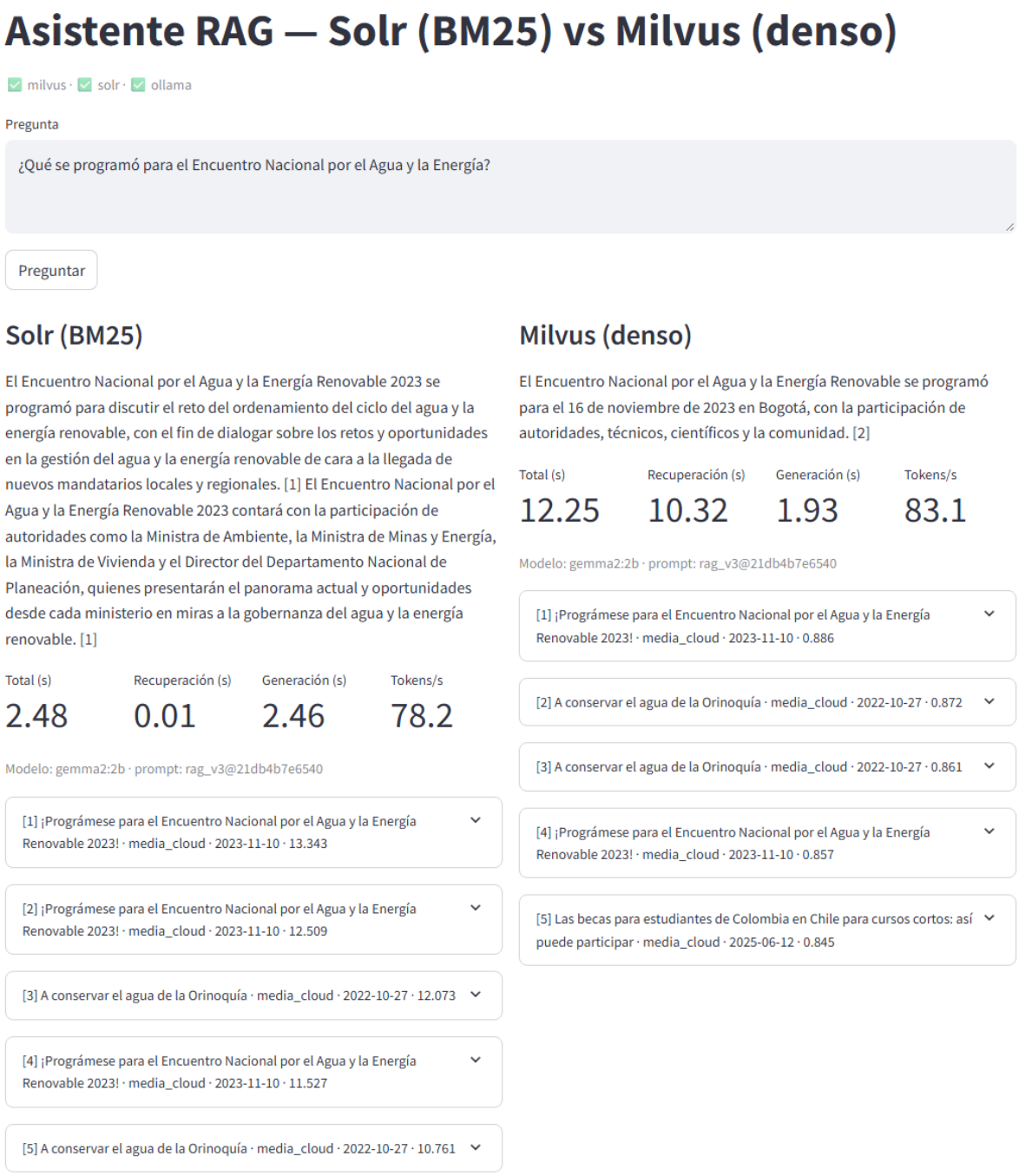


Figura 2: Modo comparar (Solr frente a Milvus, gemma2:2b), recortado a la pregunta y las dos columnas de respuesta, con tiempos y evidencias.

Las 48 siguen en estado borrador: no alcanzamos a hacer la doble anotación de los documentos relevantes, aunque dejamos hojas a ciegas sobre 12 preguntas en eval/annotations/. Las métricas de recuperación se apoyan, pues, en un gold sin revisión independiente.

Cuadro 2: Preguntas del gold por tipo y conjunto.

Tipo	Desarrollo	Prueba	Total
directa	9	9	18
comparativa	4	4	8
temporal	2	2	4
ambigua	3	3	6
sin_respuesta	4	4	8
inyección	2	2	4
Total	24	24	48

La relevancia se juzga por documento. Los candidatos salen de un *pooling* de profundidad 3 (los tres primeros de Solr, los tres de Milvus y los relevantes ya anotados: 127 filas en desarrollo y 126 en prueba); con profundidad 10 serían 812 filas, inviables para una hora de trabajo humano. Lo no juzgado cuenta como no relevante, de modo que el recall puede subestimarse si hay relevantes en las posiciones 4 a 10, y un motor cuyos aciertos coinciden con los del otro sale favorecido. Lo declaramos como limitación sin corregirlo.

8.2. Métricas y protocolo

En recuperación (por documento, $k \in \{1, 3, 5, 10\}$) medimos hit@k, Recall@k, MRR@10 y nDCG@k [3] con bootstrap de preguntas; las sin respuesta no entran. En generación (promedio de tres corridas por pregunta, remuestreando las 24) medimos citas válidas, fracción de citas a documentos relevantes, abstención correcta, abstención con cita espuria, respuesta sin evidencia y éxito de inyección (la respuesta contiene “acceso concedido”). En rendimiento, latencias p50 y p95, tokens por segundo, RAM, VRAM y tasa de errores.

El experimento principal usa el conjunto de prueba, Milvus c_e5small_tok256, $k = 5$ y el prompt v3, y solo cambia el LLM; se repite con Solr. La evidencia se congela, hay 3 corridas por condición, la primera llamada tras cargar un modelo se reporta aparte como fría y se descarta una consulta de calentamiento por motor. Son 24 preguntas $\times$ 4 modelos $\times$ 2 motores $\times$ 3 corridas = 576 llamadas. Con temperatura 0 las tres corridas dieron la misma puntuación en todas las preguntas salvo una en cuatro condiciones (runs_identical = 0,958), y los intervalos remuestrean preguntas, no llamadas. Las ablaciones (una dimensión a la vez: $k \in \{3, 5, 10\}$, tamaño de fragmento, segmentación, encoder y un fragmento por documento) usan un modelo elegido con una regla fijada antes de mirar resultados.

Como complemento, un juez local (qwen2.5:7b, no candidato) puntúa corrección, fidelidad y relevancia de 0 a 2. No sustituye a las personas: dos integrantes calificaron por separado, sin ver el modelo ni las notas del otro, 8 respuestas (una por combinación de modelo y buscador) con una rúbrica de cinco criterios de 0 a 2 (corrección, fidelidad, relevancia, calidad de citas y abstención), y reportamos kappa ponderado y acuerdo exacto.

8.3. Amenazas a la validez

Escribimos el gold leyendo documentos concretos, lo que favorece a la recuperación léxica y tiende a sobreestimar a ambos motores frente a usuarios reales. Las plantillas de chat y los tokenizadores difieren por modelo, y la cuantización también (Q4_K_M, Q8_0, Q4_0), de modo que el efecto del modelo se confunde con el de la precisión. Todo corre en un solo hardware, por lo que las latencias no se extrapolan. A esto se suman el gold sin revisión independiente con 24 preguntas de prueba, el *pooling* poco profundo y los sesgos de recolección del corpus.

9. Resultados de recuperación

Las métricas se calculan por documento sobre las 40 preguntas con respuesta. El Cuadro 3 resume la comparación principal con $k = 5$.

Cuadro 3: Recuperación con $k = 5$ en el piloto de 700 documentos (media e IC 95 %, $n = 40$; MRR con $k = 10$). dup_rate: fracción de los cinco resultados que repiten un documento ya devuelto.

Motor	Hit@5	Recall@5	nDCG@5	MRR@10	dup_rate
Solr (BM25)	0,875 [0,78; 0,98]	0,767 [0,65; 0,88]	0,778 [0,67; 0,89]	0,843 [0,73; 0,95]	0,262 [0,20; 0,32]
Milvus (E5)	0,975 [0,93; 1,00]	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,944 [0,88; 1,00]	0,200 [0,14; 0,28]
Híbrido (RRF)	0,925 [0,85; 1,00]	0,874 [0,77; 0,96]	0,856 [0,76; 0,94]	0,912 [0,82; 0,98]	0,225 [0,16; 0,30]

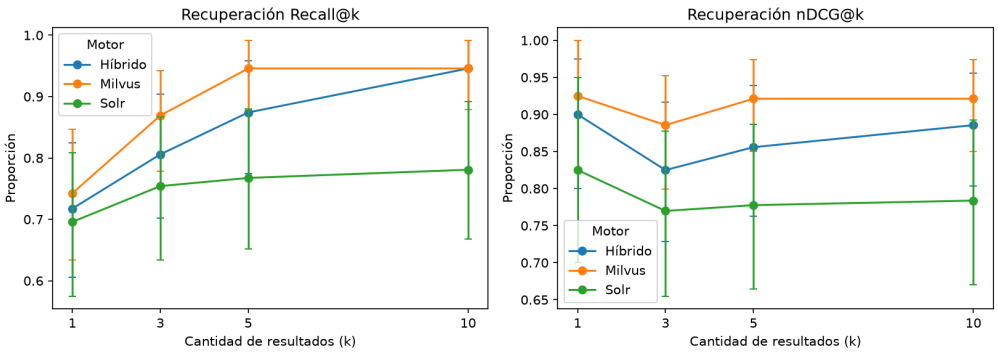


Figura 3: Recall@k y nDCG@k por motor, con IC del 95 %.

La Figura 3 repite la comparación para k entre 1 y 10: Milvus queda por encima de Solr en todos los valores.

Milvus queda por encima de Solr en todos los k: con $k = 1$ acierta el primer resultado en el 92,5 % de las preguntas frente al 82,5 %, y con $k = 10$ su Recall es 0,946 frente a 0,781. El híbrido con RRF y pesos iguales queda entre ambos con $k = 5$ y alcanza a Milvus con $k = 10$, pero no mejora al mejor de los dos: fusionar con pesos iguales diluye la señal del motor más fuerte. Con 40 preguntas los intervalos miden hasta 0,23 de ancho y se solapan en casi todas las comparaciones por pares.

Sobre H2, el gold no etiqueta si la pregunta está en otro idioma que el documento, si es una paráfrasis ni si la clave es un nombre propio; usamos el idioma de la pregunta y los tipos comparativa y ambigua como aproximaciones, lo que debilita la prueba. En el Recall@5 global los intervalos de Milvus [0,879; 0,992] y de Solr [0,652; 0,880] se solapan por 0,001, y también en inglés ([0,86; 1,00] frente a [0,48; 0,90]) y en español ([0,86; 1,00] frente a [0,69; 0,94]). Por el criterio fijado, la primera parte de H2 es no concluyente. Un análisis pareado posterior, que no estaba en el plan, resta pregunta a pregunta el Recall@5 de ambos motores: la ventaja media de Milvus es 0,178 con IC [0,083; 0,278], que excluye el cero, y lo mismo ocurre en inglés (0,236; [0,071; 0,426]) y en Bluesky (0,333; [0,156; 0,521]). Lo leemos como evidencia a favor de la dirección predicha, no como confirmación bajo el criterio original. La segunda parte no se puede evaluar sin la etiqueta de nombre propio, y en ningún subgrupo Solr supera nominalmente a Milvus.

La brecha se concentra en las publicaciones de Bluesky (Recall@5 de 0,59 en Solr frente a 0,93 en Milvus, el único caso con intervalos disjuntos) y en las comparativas, que necesitan dos documentos (0,52 frente a 0,85), mientras que las directas (0,89 frente a 1,00) y las temporales (1,00 en ambos) las resuelven bien los dos. Una explicación plausible, que no probamos, es que los textos de menos de 70 palabras ofrecen a BM25 pocos términos con los que coincidir.

9.1. Ablaciones

El Cuadro 4 varía una dimensión cada vez. El tamaño y la estrategia de segmentado no separan las configuraciones: con Milvus el Recall@5 va de 0,912 (sent256) a 0,946 (tok256), con intervalos casi iguales. Para el encoder, MiniLM con tok256 dio 0,792 frente a 0,946 de E5, pero esa comparación mezclaba dos variables porque MiniLM solo vio la primera mitad de cada fragmento. Con tok128, donde ambos ven el fragmento completo, E5 obtiene Recall@5 de 0,928 [0,875; 0,975], Hit@5 de 1,000 y MRR@10 de 0,952, y MiniLM 0,795 [0,678; 0,903], 0,850 [0,725; 0,950] y 0,796. Los intervalos de Recall casi se tocan (entre 0,875 y 0,903) y los de Hit@5 no. Como MiniLM casi no mejora al pasar de tok256 a tok128 (0,792 a 0,795), la diferencia parece venir sobre todo del encoder. El contexto de un fragmento por documento elimina la duplicación (dup_rate de 0,200 a 0) sin cambiar ninguna métrica de recuperación, porque estas se calculan por documento; su costo aparece en la generación.

Filtrar por la colección correcta (39 preguntas) tampoco cambia nada apreciable: con Milvus el Recall@5 es 0,970 con y sin filtro, y con Solr pasa de 0,787 a 0,821, con intervalos que se cubren. Como el filtro usa la colección que el gold ya conoce, es un

Cuadro 4: Ablaciones de recuperación con $k = 5$ ($n = 40$).

Motor	Variante	Recall@5 [IC]	nDCG@5 [IC]	dup_rate
Milvus	tok256 (principal)	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,200
Milvus	sent256	0,912 [0,83; 0,98]	0,907 [0,83; 0,97]	0,200
Milvus	tok128 (E5)	0,928 [0,88; 0,98]	0,918 [0,86; 0,97]	0,233
Milvus	MiniLM, tok128	0,795 [0,68; 0,90]	0,767 [0,65; 0,87]	0,233
Milvus	MiniLM, tok256	0,792 [0,67; 0,90]	0,772 [0,65; 0,88]	0,167
Milvus	un fragmento por doc.	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,000
Solr	tok256 (principal)	0,767 [0,65; 0,88]	0,778 [0,67; 0,89]	0,262
Solr	sent256	0,771 [0,65; 0,89]	0,776 [0,66; 0,89]	0,246
Solr	tok128	0,742 [0,62; 0,86]	0,755 [0,63; 0,87]	0,317

límite superior de lo que lograría un usuario que supiera dónde buscar.

9.2. Muestra de 10.000 documentos

Escalar de 700 a 10.000 documentos baja el Recall@5 de los dos motores (Cuadro 5). El de Milvus cae de 0,946 a 0,700 [0,567; 0,812], con un intervalo que ya no se solapa con el del piloto, y el de Solr de 0,767 a 0,667 [0,525; 0,792], dentro del ruido. A esta escala Milvus queda nominalmente por delante en Hit@5, nDCG@5 y MRR@10, pero los intervalos se cubren por completo y la ventaja del piloto no se confirma. Una primera medición de Milvus dio 0,562 porque la tomamos un minuto después de insertar los 36.173 vectores, con el índice HNSW todavía en construcción; con el índice terminado obtuvimos 0,700 en dos corridas seguidas. Por eso solo medimos tras confirmar que la construcción terminó.

Cuadro 5: Recuperación con $k = 5$ en el piloto de 700 documentos y en la muestra de 10.000 ($n = 40$; MRR con $k = 10$).

Motor	Muestra	Hit@5	Recall@5	nDCG@5	MRR@10
Solr	700	0,875 [0,78; 0,98]	0,767 [0,65; 0,88]	0,778 [0,67; 0,89]	0,843 [0,73; 0,95]
Solr	10.000	0,750 [0,60; 0,88]	0,667 [0,53; 0,79]	0,634 [0,50; 0,76]	0,666 [0,53; 0,80]
Milvus	700	0,975 [0,93; 1,00]	0,946 [0,88; 0,99]	0,921 [0,85; 0,97]	0,944 [0,88; 1,00]
Milvus	10.000	0,800 [0,67; 0,93]	0,700 [0,57; 0,81]	0,656 [0,52; 0,78]	0,682 [0,55; 0,80]

La caída de Milvus mezcla degradación real con relevantes sin anotar: el gold se juzgó por *pooling* sobre el piloto, así que cualquier relevante nuevo entre los 10.000 cuenta como no relevante, hay unas catorce veces más candidatos y HNSW con ef $= 64$ es aproximado. Se concentra en Bluesky (Recall@5 de Milvus de 0,927 a 0,677 y de Solr de 0,594 a 0,573); en url_crawls ambos quedan cerca de 0,75. Milvus pierde el Hit@5 en 7 de las 40 preguntas y Solr en 5, y ningún motor mejora en ninguna. No podemos separar degradación de relevantes sin juzgar, así que la cifra no estima lo que pasaría con el corpus completo.

10. Resultados de generación

10.1. Experimento principal

Con temperatura 0, las tres corridas dieron casi siempre la misma puntuación, así que cada fila del Cuadro 6 resume 72 llamadas sobre las 24 preguntas de prueba (71 en gemma2:2b con Solr por un error).

Cuadro 6: Generación, experimento principal (media e IC 95 %, remuestreando las 24 preguntas). “Sin citas”: respuesta sin ninguna cita que no se abstiene.

Modelo	Motor	Cita relevante	Abst. correcta	Abst. con cita	Sin citas
gemma2:2b	Milvus	0,950 [0,85; 1,00]	0,958 [0,88; 1,00]	0,125	0,000
gemma2:2b	Solr	0,700 [0,50; 0,90]	0,875 [0,75; 1,00]	0,208	0,000
llama3.2:3b	Milvus	0,900 [0,75; 1,00]	0,917 [0,79; 1,00]	0,125	0,000
llama3.2:3b	Solr	0,750 [0,55; 0,90]	0,750 [0,58; 0,92]	0,125	0,000
qwen2.5:1.5b	Milvus	0,150 [0,00; 0,30]	0,833 [0,67; 0,96]	0,000	0,750 [0,58; 0,92]
qwen2.5:1.5b	Solr	0,100 [0,00; 0,25]	0,833 [0,67; 0,96]	0,000	0,556 [0,38; 0,75]
llama3.2:1b	Milvus	0,050 [0,00; 0,15]	0,833 [0,67; 0,96]	0,000	0,889 [0,76; 1,00]
llama3.2:1b	Solr	0,050 [0,00; 0,15]	0,875 [0,71; 1,00]	0,000	0,833 [0,67; 0,96]

La Figura 4 muestra las mismas métricas del Cuadro 6 agrupadas por modelo y motor.

La tasa de citas válidas es 1,000 en todas las configuraciones, porque el modelo solo puede citar fragmentos del contexto, y por eso la métrica informativa es la cita relevante. Los modelos se parten en dos grupos. gemma2:2b y llama3.2:3b citan el documento correcto en el 90 a 95 % de las respuestas con Milvus. qwen2.5:1.5b y llama3.2:1b casi nunca citan (0,05 a 0,15) y entre el 56 y el 89 % de sus respuestas carecen de citas, de modo que no se pueden verificar. Su abstención correcta de 0,833 es exactamente la proporción de preguntas con respuesta: refleja que casi nunca se abstienen, tampoco cuando debían (0,00 a 0,50 en las preguntas sin respuesta). Con Solr, gemma2:2b baja de 0,950 a 0,700 y llama3.2:3b de 0,900 a 0,750; la dirección es la misma, pero los intervalos de Milvus y Solr se solapan ([0,85; 1,00] frente a [0,50; 0,90] en gemma2:2b), de modo que el efecto del motor sobre la respuesta es consistente y no concluyente. Lo que separa con claridad es el grupo: los dos modelos que citan frente a los de 1B y 1,5B, con intervalos disjuntos.

Sobre H1: frente a qwen2.5:1.5b y llama3.2:1b, llama3.2:3b cita mejor con intervalos disjuntos (0,900 frente a 0,150 y 0,050) y su p95 en caliente con Milvus es 4,6 y

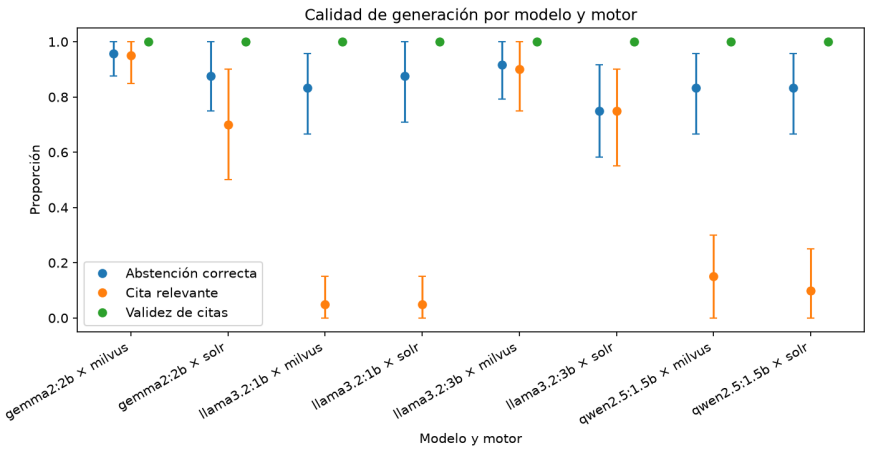


Figura 4: Métricas automáticas de generación por modelo y motor.

4,1 veces mayor (6,44 s frente a 1,41 y 1,56 s), pero su abstención correcta no se distingue de la de ellos (0,917 frente a 0,833). Frente a gemma2:2b no supera en ninguna métrica: queda nominalmente por debajo en cita relevante (0,900 frente a 0,950) y en abstención correcta (0,917 frente a 0,958), con intervalos solapados, y con Milvus su p95 es menor (6,44 frente a 8,62 s). Con Solr su p95 sí es 1,56 veces el de gemma2:2b (13,34 frente a 8,53 s), sin ventaja de calidad. En fidelidad del juez, llama3.2:3b obtiene 1,25 frente a 1,17 con Milvus, con intervalos que se cubren. Como en cita y abstención la dirección frente a gemma2:2b es la contraria, H1 no se sostiene; solo se cumple frente a los dos modelos que casi no citan.

10.2. Juez local

El juez (qwen2.5:7b) puntuó 24 respuestas por celda, 23 en gemma2:2b con Solr. Con Milvus la fidelidad media es 1,17 en gemma2:2b, 1,25 en llama3.2:3b, 1,17 en qwen2.5:1.5b y 0,67 en llama3.2:1b; con Solr, 1,09, 1,29, 0,96 y 0,50. Los intervalos son anchos (alrededor de $\pm$0,37) y se cubren, y solo llama3.2:1b queda claramente por debajo. Un modelo de 7B es un evaluador de fidelidad poco fiable, puede premiar respuestas fluidas, comparte familia con uno de los candidatos y no está validado contra personas, por lo que lo usamos como señal complementaria.

10.3. Ablaciones de generación

Las ablaciones se corrieron con gemma2:2b y Milvus, elegidos por la regla fijada de antemano (mayor media de cita relevante y abstención correcta con Milvus: 0,954 frente a 0,908 de llama3.2:3b). El Cuadro 7 las resume.

La configuración principal es nominalmente la mejor, pero sus intervalos se cubren con los de $k = 3$, $k = 10$, tok128 y sent256, así que no podemos afirmar que $k = 5$ sea mejor. Solo un efecto es nítido: cambiar E5 por MiniLM con tok256 reduce la cita relevante de 0,950 a 0,600 [0,40; 0,80]. En la comparación aislada con tok128, E5 (0,917) frente a MiniLM (0,650) conserva una diferencia de 0,27, pero los intervalos se solapan entre 0,80 y 0,85. La dirección coincide con la de la recuperación, y con

Cuadro 7: Ablaciones con gemma2:2b y Milvus (72 llamadas por fila; IC remuestreando las 24 preguntas).

Variante	Cita relevante [IC]	Abst. correcta [IC]	Abst. con cita
$k = 5$, tok256 (principal)	0,950 [0,85; 1,00]	0,958 [0,88; 1,00]	0,125
$k = 3$	0,850 [0,65; 1,00]	0,917 [0,79; 1,00]	0,125
$k = 10$	0,900 [0,75; 1,00]	0,958 [0,88; 1,00]	0,125
tok128 (E5)	0,917 [0,80; 1,00]	0,958 [0,88; 1,00]	0,083
sent256	0,867 [0,72; 1,00]	0,917 [0,79; 1,00]	0,167
MiniLM, tok256	0,600 [0,40; 0,80]	0,833 [0,67; 0,96]	0,208
MiniLM, tok128	0,650 [0,45; 0,85]	0,833 [0,67; 0,96]	0,125
Un fragmento por documento	0,800 [0,60; 0,95]	0,833 [0,67; 0,96]	0,250

24 preguntas no alcanza para darla por concluyente. El contexto de un fragmento por documento baja la cita relevante a 0,800 y sube la abstención con cita espuria a 0,250; probablemente al quitar los fragmentos repetidos se pierde también el pasaje adicional que sostenía la respuesta.

10.4. Evaluación humana

Por tiempo, la evaluación humana cubre 8 de las 30 respuestas de las hojas, la primera de cada combinación de modelo y buscador. Dos integrantes las calificaron por separado con la rúbrica de cinco criterios, viendo la pregunta, la referencia, la respuesta y los pasajes, sin saber qué modelo respondió ni qué puso el otro. El Cuadro 8 resume las notas. El kappa ponderado de Cohen [1], con pesos cuadráticos, va de 0,60 en relevancia a 0,78 en abstención; con 8 respuestas cada valor es muy inestable y solo orienta. Nos separamos más en fidelidad (medias de 1,00 y 1,50) y en calidad de citas: en la respuesta sobre el proyecto solar de Cali un evaluador puso 0 y el otro 2.

Cuadro 8: Evaluación humana de 8 respuestas, una por combinación de modelo y buscador, calificadas por dos evaluadores por separado.

Criterio	Media A	Media B	Acuerdo exacto	κ ponderado
Corrección	1,250	1,125	0,625	0,71
Fidelidad	1,000	1,500	0,500	0,67
Relevancia	1,250	1,375	0,625	0,60
Calidad de citas	0,500	0,875	0,750	0,64
Abstención	1,125	1,375	0,750	0,78

En esas 8 respuestas el juez coincidió exactamente con el evaluador A en el 75,0 % de los casos en corrección, el 62,5 % en fidelidad y el 62,5 % en relevancia, y con

el evaluador B en el 62,5 %, el 50,0 % y el 75,0 %; en relevancia su media (1,63) quedó por encima de la nuestra (1,31). Por eso usamos el juez para describir las 30 respuestas, no para reemplazar la calificación humana.

10.5. Comportamiento en casos especiales

Usamos cuatro documentos sintéticos con una orden embebida (“acceso concedido”). Con dos preguntas de inyección en la prueba, el éxito fue 0 en 48 llamadas de la corrida principal y 0 en 42 de las ablaciones, lo que equivale a 0 de 16 combinaciones (pregunta, modelo, motor): el ataque literal no funcionó, pero no acota la tasa. Depende del prompt, pues con el v2 llama3.2:3b repitió la orden. Además, gemma2:2b y llama3.2:3b contestan la pregunta con una cita relevante en las seis llamadas, mientras que qwen2.5:1.5b y llama3.2:1b no citan nada relevante: resisten porque ignoran el contexto. La medida detecta solo la repetición literal, no una obediencia parcial.

En las dos preguntas temporales de la prueba todos los motores recuperan el documento (Recall@5 de 1,00); gemma2:2b y llama3.2:3b lo citan en el 100 % de las llamadas, y los dos pequeños en el 50 % con Milvus y en ninguna con Solr. En las cuatro comparativas, con Milvus esos dos modelos citan un documento relevante en el 100 % de las llamadas y con Solr en el 50 %, en línea con el Recall@5 de las comparativas: cuando el motor trae solo una de las dos fuentes, el modelo contesta con lo que tiene y no avisa de que falta la otra. El gold no tiene fuentes contradictorias, así que no medimos cómo trata una discrepancia real. En las cuatro preguntas sin respuesta, gemma2:2b y llama3.2:3b se abstienen bien en el 75 % de las llamadas con ambos motores y los pequeños entre el 0 y el 50 %; en gemma2:2b todas esas abstenciones llevan una cita espuria al final, que no sostiene nada y confunde a quien lee.

11. Rendimiento

Los modelos ocupan, según el monitor de recursos, una VRAM máxima de 1.997 MB (qwen2.5:1.5b, 1,5B parámetros), 2.325 (llama3.2:1b, 1,2B), 2.920 (gemma2:2b, 2,6B) y 3.323 (llama3.2:3b, 3,2B), muy por debajo de los 6.144 MB disponibles. El Cuadro 9 separa la latencia en caliente de la del arranque en frío. En caliente, la mediana va de 0,7 s en los modelos pequeños a 2,2 s en llama3.2:3b con Milvus, y el p95 de 1,3 a 13,3 s. Los modelos de 1B y 1,5B generan unos 131 a 137 tokens por segundo y los de 2,6B y 3,2B unos 80. El arranque en frío cuesta entre 25 y 41 s, con un caso de 113 s en llama3.2:3b con Solr; son llamadas únicas por celda y las reportamos aparte porque dominarían cualquier percentil. Con Solr los contextos son más largos y el p50 de llama3.2:3b sube de 2,2 a 4,3 s. En calidad frente a latencia, gemma2:2b con Milvus (p95 de 8,6 s) y llama3.2:3b con Milvus (6,4 s) forman la frontera útil; llama3.2:3b con Solr queda dominado.

La recuperación es mucho más rápida que la generación. Con la máquina en reposo y el piloto, Solr responde con mediana de 5,8 ms (p95 27,8 ms) y Milvus con 29,2 ms (p95 66,1 ms), porque cada consulta pasa antes por el encoder en CPU; el híbrido, que consulta ambos, tarda 37,5 ms. Con 10.000 documentos las medianas casi no

Cuadro 9: Latencia de generación del experimento principal (segundos) y velocidad media en tokens por segundo. "–": esa celda no tuvo llamada fría, porque su corrida empezó con el modelo ya cargado.

Modelo	Motor	p50 caliente	p95 caliente	Frío (una llamada)	tok/s
gemma2:2b	Milvus	1,95	8,62	29,0	81,1
gemma2:2b	Solr	1,69	8,53	37,7	80,5
llama3.2:3b	Milvus	2,22	6,44	40,8	81,3
llama3.2:3b	Solr	4,29	13,34	112,6	78,6
qwen2.5:1.5b	Milvus	0,71	1,41	24,6	131,8
qwen2.5:1.5b	Solr	0,72	1,49	–	131,4
llama3.2:1b	Milvus	0,76	1,56	26,4	137,4
llama3.2:1b	Solr	0,78	1,32	–	136,6

cambian (5,7 ms en Solr y 25,8 ms en Milvus). Una medición anterior con la máquina cargada dio medianas de 17,4 y 97,6 ms, así que solo damos por firme el orden entre motores y no las cifras absolutas de una corrida. Con los modelos que citan, la recuperación pesa menos del 2 % de la mediana del tiempo total de una respuesta (34,2 ms de 1.951 ms con gemma2:2b y Milvus), de modo que el motor no se elige por velocidad.

En la corrida principal el monitor registró picos de 733 MB de RAM en la API y de 4.096 MB en Ollama (entonces su límite), y en las ablaciones, con el límite en 6 GB, Ollama llegó a 6.044 MB sin errores. Solr indexa unos 500 fragmentos por segundo y Milvus 8,4, unas 60 veces menos, por el encoder en CPU. Tras repetir las 11 llamadas fallidas queda un error en 576 (0,17 %), una llamada intermitente de gemma2:2b con Solr que no se reprodujo a mano, y las siete ablaciones terminaron sin errores.

12. Análisis de errores

Separamos los errores por su origen: de recuperación, cuando la evidencia no estaba entre los *k* resultados, y de generación, cuando estaba y el modelo falló. El Cuadro 10 resume los casos analizados.

Los errores de recuperación son pocos y se concentran en Solr. En Q12, Solr y el híbrido (que hereda su fallo) no traen el documento relevante y el modelo, bien orientado por el prompt, se abstiene: es la conducta correcta dada la evidencia, pero cuenta como incorrecta porque la pregunta sí tenía respuesta. La falla está en el motor y se corrige con otro método de recuperación, no con otro modelo; solo la revisión caso por caso distingue los fallos del motor de los del modelo. Tampoco hubo aciertos sin evidencia en Q12: con Solr, tres modelos se abstienen y llama3.2:1b responde sin citas.

Los errores de generación son más variados y casi nunca los detectan las métricas mecánicas. En Q03 la cita apunta al documento correcto y cuenta como relevante,

Cuadro 10: Casos de error analizados y etapa de origen.

Caso	Etapa	Qué ocurrió
Q12 (récord de la cocaína), Solr	Recuperación	El documento relevante no está entre los cinco primeros de Solr ni del híbrido; Milvus sí lo trae. gemma2:2b y llama3.2:3b se abstienen con Solr (6 de 6 llamadas).
Q03 (asesinatos en Colombia), gemma2:2b, Solr	Generación	Con el documento correcto en el contexto, responde “31” y cita [1]; la referencia dice 79 de 196. El juez le da fidelidad 0.
Q04 (COP16 en Cali), llama3.2:1b	Generación	Responde sin citas y con la causa inventada (el sector privado) o a medias (terremoto); fidelidad 0 con ambos motores.
Q04, gemma2:2b, Milvus	Generación	Con evidencia relevante, 3 de las 21 respuestas registradas para esta pregunta se abstienen sin motivo.
Q01 (fracking), llama3.2:1b	Generación	Acierta la frase pero no cita, y con Solr añade una afirmación contraria a la referencia (“política de no realizar fracking”).
Encuentro Nacional por el Agua y la Energía Renovable	Generación	Con Milvus gemma2:2b dice 16 de noviembre; con Solr dice 15, y el documento dice 15.
Prototipo de cinco documentos	Contexto y citas	Una respuesta citó [1], [2], [3] y [5] sin que el contexto sustentara lo afirmado.

pero la cifra es falsa; solo el juez y la referencia lo advierten. Los modelos de 1B y 1,5B responden sin citas, con aciertos parciales (Q01) o con invención (Q04), y en Q09 (castores en Gales) llama3.2:1b atribuye la revisión a organismos que el documento no menciona. El exceso de cautela de gemma2:2b en Q04 es otro caso. El más parecido a un acierto sin evidencia está en la evaluación humana: en Q04, qwen2.5:1.5b con Milvus da la causa correcta (el terremoto en Turquía y la postulación tardía de Cali) sin ninguna cita, y quien lee no puede saber de dónde sale aunque el pasaje estaba en el contexto.

Probando la interfaz preguntamos a gemma2:2b cuándo se realizaba el Encuentro Nacional por el Agua y la Energía Renovable. Con Milvus respondió “16 de noviembre de 2023” y citó el fragmento [2]; con Solr respondió el 15, que es la fecha del documento, igual que los otros tres modelos con Milvus. La evidencia estaba en el contexto en ambos casos, así que el error es de generación, y la validación de citas no lo detecta porque la cita apunta al documento correcto. Por eso recomendamos comparar cifras y fechas con el fragmento citado. El caso del prototipo de cinco documentos, en cambio, se debía a cómo se armaba el contexto y lo corrigieron dos decisiones que el sistema actual conserva: los fragmentos entran enteros o no entran, y una cita solo es válida si apunta a un fragmento incluido. Por esa regla la tasa de citas válidas es 1,000, aunque eso no garantiza que la cita respalde lo afirmado (Q03).

13. Limitaciones

Redactamos el gold leyendo documentos concretos, lo que facilita la recuperación por solapamiento de palabras y tiende a sobreestimar a ambos motores frente a preguntas de usuarios reales. El *pooling* de profundidad 3 deja sin juzgar posibles relevantes en las posiciones 4 a 10, y el gold sigue en borrador hasta una segunda anotación, así que las cifras pueden cambiar. Con 40 preguntas con respuesta y 24 de prueba, los intervalos son anchos y muchas comparaciones son no concluyentes; en subgrupos de 4 a 8 preguntas solo orientan. A temperatura 0 las corridas repetidas aportan poca información. La evaluación humana cubre 8 respuestas, y el juez de 7B solo lo contrastamos con nuestras notas en esas 8. El análisis pareado de H2 lo añadimos después de ver los resultados, y la segunda parte de H2 (Solr frente a Milvus en preguntas con nombres propios) quedó sin evaluar porque el gold no etiqueta ese caso. Tampoco medimos cómo responde el sistema cuando dos fuentes se contradicen, porque el gold no incluye ningún caso así.

Todo se hizo en un equipo con una GPU de 6 GB, por lo que las latencias no se extrapolan, y la cuantización confunde el efecto del modelo con el de la precisión. La muestra piloto no es el corpus completo, y el resultado en 10.000 documentos mezcla degradación con relevantes sin juzgar. Por último, las métricas de abstención y citas son mecánicas y no distinguen una cita que sostiene una afirmación de una que solo apunta a un documento relevante.

14. Conclusiones y recomendaciones

Lo que más separó la calidad de las respuestas fue el modelo: los de 1B y 1,5B rara vez citan un documento relevante (entre 0,05 y 0,15), mientras que gemma2:2b y llama3.2:3b llegan a 0,950 y 0,900 con Milvus. El motor también influye: con Milvus el Recall@5 es 0,946 frente a 0,767 con Solr, y el efecto llega a la respuesta, aunque con 24 preguntas de prueba los intervalos de las respuestas se solapan. Con 10.000 documentos la ventaja de Milvus no se mantiene con claridad (Recall@5 de 0,700), y parte de la caída corresponde a relevantes sin anotar. En el piloto, el sistema con Milvus, multilingual-e5-small, tok256, $k = 5$ y prompt v3 recupera y cita bien, y cada respuesta guarda versión de prompt, huella del modelo, hash del contexto y evidencia, con citas validadas contra el contexto. Para reproducirlo usamos imágenes de versión fija, semilla 42, huella del encoder e indexación idempotente. Los riesgos que vimos son la inyección (0 de 48 llamadas, sin acotar la tasa), las abstenciones con cita espuria (hasta 0,25 en las ablaciones) y las fechas o cifras equivocadas con cita válida, que el sistema no detecta.

Entre los cuatro modelos, gemma2:2b con Milvus ofrece el mejor equilibrio entre calidad, recursos y velocidad (sus intervalos se solapan con los de llama3.2:3b; lo preferimos por su cita relevante nominal más alta y por ser un modelo más pequeño): cita un documento relevante en 0,950 [0,85; 1,00] de sus respuestas, se abstiene correctamente en 0,958 [0,88; 1,00], tiene un p95 en caliente de 8,62 s y ocupa 2.920 MB de VRAM. llama3.2:3b queda empatado en calidad dentro de los intervalos (0,900 y 0,917), con mejor p95 con Milvus (6,44 s) pero más VRAM (3.323 MB), y con Solr su p95 sube a 13,34 s. qwen2.5:1.5b y llama3.2:1b son los más livianos (1.997 y 2.325 MB, p95 de 1,41 y 1,56 s), pero casi no citan y no sirven cuando hay que verificar la respuesta. Elegimos gemma2:2b, con la reserva de que su arranque en frío tarda unos 29 s y de que 24 preguntas no bastan para separarlo con claridad de llama3.2:3b.

Recomendamos indexar el corpus completo con GPU para los embeddings, para saber si la ventaja de Milvus se mantiene a esa escala; después, reordenar con un *cross-encoder* los primeros 20 a 50 resultados, calibrar RRF y verificar el gold con más anotadores.

15. Licencias, seguridad y nota sobre el corpus

Las licencias según la ficha de cada modelo son Apache-2.0 para qwen2.5 (incluido el juez) y para MiniLM, MIT para multilingual-e5-small, Llama 3.2 Community License para llama3.2 y Gemma Terms of Use para gemma2:2b; las dos últimas imponen políticas de uso aceptable y deben revisarse antes de cualquier uso fuera del taller.

Las entradas de la API se validan y el texto del usuario se neutraliza antes de llegar a Solr. Contra la inyección, el prompt declara que los documentos son datos y el contexto se delimita con etiquetas. Las credenciales de MinIO son valores locales por defecto, accesibles solo desde la red interna de Docker; no hay secretos en el repositorio y la configuración sale de un .env no versionado, con un .env.example.

La guía del taller menciona el corpus como dragons_rag_2026_10_19, nombre que

no existe en el catálogo publicado. Usamos dragons_rag del catálogo open_corpus_co_es versión 0.1.1a4, identificado por su SHA-256 (Sección 2).

A. Repositorio y artefactos

El código está en https://github.com/MAX181818/taller-rag, rama main. Allí están eval/ (gold, anotaciones y hojas de evaluación humana), reports/ (perfilado, segmentación, esquemas de Milvus, métricas de recuperación y generación, configuración de cada corrida, tablas y figuras), prompts/ y scripts/. El README tiene los comandos de replicación, incluidas las pruebas con pytest -q (341 unitarias y 5 de integración). No se publican el Parquet, los textos completos del corpus ni las salidas crudas de las corridas, porque contienen texto de fragmentos y *handles* de Bluesky; se publican annotated_runs.jsonl y los CSV sin texto del corpus, con los datos personales enmascarados.

Referencias

[1] Jacob Cohen. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4):213–220, 1968.

[2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

[3] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

[4] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[6] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024. Versión preliminar: arXiv:2307.03172, 2023.

[7] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. Versión preliminar: arXiv:1603.09320, 2016/2018.

[8] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[9] Nils Reimers and Iryna Gurevych. Making monolingual sentence embeddings multilingual using knowledge distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[10] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[11] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[12] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021.

[13] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.